

Workshop 6: Responsive Design Workshop

Asynchronous JavaScript, Promise, Async/Await และ Fetch API

เป้าหมาย

Workshop นี้ให้นักศึกษาฝึกการทำงานแบบ Asynchronous ของ JavaScript โดยเน้นหัวข้อต่อไปนี้

- Synchronous และ Asynchronous
- Callback Function
- Promise
- .then() และ .catch()
- async และ await
- Fetch API

นักศึกษาต้องสร้างไฟล์ทั้งหมด 3 ไฟล์

1. async-demo.html
2. promise-download.html
3. fetch-users.html

งานที่ 1 — Asynchronous Task

ชื่อไฟล์: async-demo.html

สร้างโปรแกรมจำลองการทำงานที่ใช้เวลานาน โดยใช้ `setTimeout()`
เมื่อกดปุ่ม **Start Task** ให้แสดงข้อความตามลำดับดังนี้

Start

Working...

End

โดยข้อความ Working... ต้องแสดงหลังจากผ่านไป 3 วินาที

HTML

```
<h1>Async Task</h1>

<button onclick="startTask()">
  Start Task
</button>

<p id="result"></p>
```

JavaScript

```
function startTask() {

  let result =
    document.getElementById("result");

  result.innerHTML = "Start<br>";

  setTimeout(function () {

    result.innerHTML +=
      "Working...<br>";

  }, 3000);

  result.innerHTML += "End<br>";
}
```

ทดลองรันโปรแกรม

ให้นักศึกษาสังเกตว่า Output ที่เกิดขึ้นจริงคือ

Start

End

Working...

คำถาม:

เพราะเหตุใดข้อความ End จึงแสดงก่อน Working...

แนวคิด: JavaScript ไม่รอ setTimeout() ให้ทำงานเสร็จก่อน แต่จะทำคำสั่งถัดไปต่อทันที

สิ่งที่ต้องใช้

- Function
- `setTimeout()`
- Callback Function
- DOM
- `innerHTML`

งานที่ 2 — Download Simulation with Promise

ชื่อไฟล์: promise-download.html

สร้างโปรแกรมจำลองการ Download ข้อมูล โดยใช้ JavaScript Promise

เมื่อกดปุ่ม **Download** ให้โปรแกรมรอ 3 วินาที จากนั้นแสดงข้อความ

Download completed!

HTML

```
<h1>Download Simulation</h1>

<button onclick="startDownload()">
  Download
</button>

<p id="result"></p>
```

สร้าง Promise

สร้าง Function ชื่อ

download()

```
function download() {

  return new Promise(function(resolve, reject) {

    setTimeout(function() {

      resolve("Download completed!");

    }, 3000);

  });
}
```

Promise จะอยู่ในสถานะ

- **Pending** — กำลังทำงาน
- **Fulfilled** — ทำงานสำเร็จ
- **Rejected** — เกิดข้อผิดพลาด

ใช้ `.then()`

สร้าง Function

```
function startDownload() {  
  
    document.getElementById("result").innerHTML =  
        "Downloading...";  
  
    download()  
        .then(function(message) {  
  
            document.getElementById("result").innerHTML =  
                message;  
  
        });  
}
```

เมื่อกดปุ่มควรแสดง

Downloading...

และหลังจาก 3 วินาที เปลี่ยนเป็น

Download completed!

Challenge — Error Handling

ปรับ Promise ให้สามารถเกิด Error ได้ และใช้ `.catch()` จัดการ Error
ตัวอย่าง

```
download()  
    .then(function(message) {  
  
        document.getElementById("result").innerHTML =  
            message;  
  
    })  
    .catch(function(error) {  
  
        document.getElementById("result").innerHTML =  
            error;  
  
    });
```

สิ่งที่ต้องใช้

- Promise
- resolve
- reject
- .then()
- .catch()
- setTimeout()

งานที่ 3 — Fetch Data from API

ชื่อไฟล์: fetch-users.html

สร้างหน้าเว็บสำหรับดึงข้อมูลจาก API ด้วย fetch()

เมื่อกดปุ่ม **Load Users** ให้ดึงข้อมูล User จาก

`https://jsonplaceholder.typicode.com/users`

แล้วแสดงชื่อของ User บนหน้าเว็บไซต์

HTML

```
<h1>User List</h1>

<button onclick="loadUsers()">
  Load Users
</button>

<div id="result"></div>
```

ใช้ Async/Await

สร้าง Function

```
async function loadUsers() {

  const result =
    document.getElementById("result");

  result.innerHTML = "Loading...";

  try {

    const response = await fetch(
      "https://jsonplaceholder.typicode.com/users"
    );

    const users = await response.json();

    result.innerHTML = "";

    for (let user of users) {
```

```

        result.innerHTML +=
            "<p>" + user.name + "</p>";
    }

    } catch (error) {

        result.innerHTML =
            "Cannot load data";
    }
}

```

ลำดับการทำงาน

คำสั่ง

```
const response = await fetch(url);
```

ใช้สำหรับรอ Response จาก Server
จากนั้น

```
const users = await response.json();
```

ใช้สำหรับแปลงข้อมูลที่ได้รับ ให้อยู่ในรูปแบบ JavaScript Object

ตัวอย่างผลลัพธ์

เมื่อกดปุ่ม

Load Users

หน้าเว็บไซต์จะแสดงรายชื่อ เช่น

Leanne Graham
Ervin Howell
Clementine Bauch
Patricia Lebsack
...

Challenge

ปรับโปรแกรมให้แสดงทั้ง

- Name

- Email
- Website

ตัวอย่าง

Name: Leanne Graham

Email: Sincere@april.biz

Website: hildegard.org

สิ่งที่ต้องใช้

- `async`
- `await`
- `fetch()`
- `response.json()`
- `try...catch`
- Loop
- DOM

Checklist ก่อนส่งงาน

1. Asynchronous Task

- ☐ มีไฟล์ `async-demo.html`
- ☐ มีปุ่ม Start Task
- ☐ ใช้ `setTimeout()`
- ☐ ใช้ Callback Function
- ☐ อธิบายได้ว่าเหตุใด End แสดงก่อน Working

2. Promise Download

- ☐ มีไฟล์ `promise-download.html`
- ☐ สร้าง Promise ได้
- ☐ ใช้ `resolve`
- ☐ ใช้ `.then()`
- ☐ แสดง Download completed ได้
- ☐ ใช้ `.catch()` ได้ (Challenge)

3. Fetch API

- ☐ มีไฟล์ `fetch-users.html`
- ☐ ใช้ `fetch()`
- ☐ ใช้ `async`
- ☐ ใช้ `await`
- ☐ ใช้ `response.json()`
- ☐ ใช้ `try...catch`
- ☐ แสดงข้อมูลจาก API บนหน้าเว็บได้

การทดสอบโปรแกรม

เปิดแต่ละไฟล์ด้วย Web Browser แล้วกด **F12** หรือเลือก **Inspect**
จากนั้นเปิด Tab

Console

ตรวจสอบว่าไม่มี JavaScript Error และทดสอบว่าทุกปุ่มสามารถทำงานได้จริง

Callback → Promise → Async/Await → Fetch API